

Report
Of
C++ Programming Internship at CodeAlpha

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE (AI & ML)



By
Roll Number of Student: 25SCS1003004654
Name of Student: ADITYA AGRAWAL
Batch: 2025 – 2029

2026-27

CANDIDATE'S DECLARATION

I, Aditya Agrawal, do hereby declare that the internship report titled “C++ Programming Internship at CodeAlpha” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI & ML). All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement, and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature:- *Aditya Agrawal*

Student Name: Aditya Agrawal

Roll No.: 25SCS1003004654

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout this Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to thank CodeAlpha for providing me the opportunity and the structured platform to learn and gain hands-on exposure in the domain of C++ Programming. I am also thankful to my institute, IILM University, Greater Noida, and the faculty of the School of Computer Science and Engineering for their continuous support and encouragement.

I express my gratitude to my parents for their blessings.

Signature: *Aditya Agrawal*

Student Name: Aditya Agrawal

Roll No.: 25SCS1003004654

TABLE OF CONTENTS

1. Candidate's Declaration.....	i
2. Acknowledgement.....	ii
3. Project Description.....	1
3.1 Introduction.....	1
3.2 Organization Profile.....	1
3.3 Problem Statement.....	2
3.4 Project Objectives.....	2
3.5 Scope of the Project.....	3
3.6 Technologies and Tools Used.....	4
3.7 System Architecture.....	4
3.8 Methodology.....	7
3.9 Expected Outcomes.....	8
3.10 Offer Letter and Communication Proof.....	10
4. Bibliography / References.....	11

Internship Offer Letter



YOUR DREAM OUR PASSION

INTERNSHIP OFFER LETTER

12th August 2026

STUDENT ID: CA/DF1/245935

Name: Aditya Agrawal

Welcome to CodeAlpha

Congratulations! We are delighted to make you an offer as **C++ Programming Internship**. This offer letter is to confirm your selection **C++ Programming Internship** at **CodeAlpha** and welcome you for the same effective from **20th August 2026 to 20th September 2026**. Hope you will be doing well.

This Internship is observed by CodeAlpha as being a learning opportunity for you.

Your internship will embrace orientation and give emphasis on learning new skills with a deeper understanding of concepts through hands-on application of the knowledge you gained as an intern. You will acknowledge your obligation to perform all work allocated to you to the best of your ability within lawful and reasonable direction given to you.

We look forward to a worthwhile and fruitful association which will make you equipped for future projects.

Wishing you the most enjoyable and truly meaningful internship program experience.

Sincerely,



Swati Srivastava
Founder & CEO





 **Phone**
+91 9336576683

 **Email**
services@codealpha.tech

 **Address**
Lucknow U.P

3. Project Description

3.1 Introduction

This report documents the work completed during a C++ Programming Internship undertaken with CodeAlpha, a software development company offering hands-on virtual internship programs across various technology domains. The internship was undertaken to gain practical, project-based experience in C++ programming, building on classroom learning of programming fundamentals and applying them to real, working software.

The internship required completion of at least two or three tasks selected from a set of four available C++ programming tasks. After evaluating all four tasks against criteria such as difficulty, the C++ concepts demonstrated, reliability, and portfolio value, three tasks were selected and completed in full: the CGPA Calculator, the Banking System, and the Sudoku Solver. The internship was completed over the period 20 August 2026 to 20 September 2026, as confirmed by the official CodeAlpha Offer Letter submitted along with this report. This report presents the organization profile, problem statement, objectives, scope, technologies used, system architecture, methodology, and the outcomes and learnings from this internship.

3.2 Organization Profile

CodeAlpha is a software development company that offers virtual internship programs to students across various technology domains, including C++ Programming, with a focus on innovation, performance, and scalable solutions. The C++ Programming internship track is designed to help interns master core C++ concepts, develop efficient algorithms, and build high-performance applications, while gaining practical exposure to memory management, object-oriented programming, and debugging under mentorship.

Internship Perks Offered

- Internship Offer Letter
- Completion Certificate (QR Verified)
- Unique ID Certificate
- Letter of Recommendation (based on performance)
- Job Opportunities / Placement Support

- Resume Building Support

Table 3.1: Organization and Certification Details

Website	www.codealpha.tech
Contact Email	services@codealpha.tech, services.codealpha@gmail.com
Internship Domain	C++ Programming
Offer Letter / Student ID	CA/DF1/245935
Issuing Authority	Swati Srivastava, Founder & CEO, CodeAlpha
Date of Issue	12 August 2026

3.3 Problem Statement

Students often learn programming through isolated textbook exercises that do not reflect how software is actually designed, built, tested, and documented in practice. The internship's underlying problem statement was to design and build fully working, independently testable C++ console applications that solve realistic problems, rather than short disconnected code snippets. Specifically, the three selected tasks addressed the following problems:

- **CGPA Calculator:** Manually calculating GPA and CGPA across multiple semesters and courses is repetitive and error-prone; the program automates this calculation from course grades and credit hours.
- **Banking System:** A bank needs a reliable way to manage customer accounts — handling deposits and withdrawals safely without allowing overdrafts, transferring funds between accounts, and maintaining an accurate transaction history.
- **Sudoku Solver:** Solving a Sudoku puzzle by hand is slow and error-prone for harder puzzles; the program automatically completes a partially filled 9×9 grid while respecting all Sudoku rules.

3.4 Project Objectives

- To gain practical, hands-on experience in C++ programming beyond academic coursework.

- To apply core programming concepts — loops, functions, arrays, structs — to real, working applications.
- To learn and apply object-oriented programming (OOP) principles such as encapsulation and composition in a substantial project.
- To understand and implement a classic algorithmic technique (recursive backtracking) in a real problem-solving context.
- To practice writing clean, modular, well-documented, and properly validated code suitable for real-world use.
- To gain experience with the full software development cycle: designing, coding, testing, debugging, documenting, and preparing work for GitHub submission.
- To build a portfolio of verifiable, working projects suitable for a resume and technical interviews.

3.5 Scope of the Project

In Scope

- Task 1 – CGPA Calculator: fully implemented and tested.
- Task 3 – Sudoku Solver: fully implemented and tested.
- Task 4 – Banking System: fully implemented and tested.

Out of Scope

Task 2 (Login and Registration System) was evaluated but not selected, primarily because plaintext file-based credential storage is inherently insecure for a portfolio-facing demonstration, and the file-handling concepts it would demonstrate were already covered more thoroughly by the Banking System task.

Known Limitations (by design, for a beginner-to-intermediate scope)

- CGPA Calculator: supports only standard A–F letter grades on a 4.0 scale (no +/- grades); data is not saved between runs.
- Banking System: account and transaction data exist only in memory during a single run and are not persisted to disk; one account per customer; no PIN/password protection.
- Sudoku Solver: console-only interface (a GUI was explicitly marked optional in the original task brief); supports only standard 9×9 puzzles.

3.6 Technologies and Tools Used

Table 3.2: Technologies and Tools Used

Programming Language	C++ (C++17 standard)
Compiler	g++ (GNU Compiler Collection)
Code Editor / IDE	Visual Studio Code
Version Control	Git and GitHub
Operating Environment	Linux-based terminal / command-line compilation and execution
Libraries Used	Standard C++ Library — <code><iostream></code> , <code><vector></code> , <code><string></code> , <code><iomanip></code> , <code><limits></code> , <code><ctime></code>
Documentation	Markdown (README files), Microsoft Word (this report)

3.7 System Architecture

Each project follows a simple, modular, menu-driven console architecture. Before writing any code, the official CodeAlpha task requirements were reviewed to understand the exact specification for each task. Each project was then designed around the data structures best suited to its problem, implemented function-by-function (or class-by-class) with input validation throughout, compiled with warnings enabled, and verified by actually running the compiled program with realistic sample input. The architecture of each completed project is described below.

3.7.1 CGPA Calculator – Architecture

Key C++ Concepts	Structs, vectors, functions, loops, pass-by-reference, input validation, formatted output (<code>iomanip</code>)
Core Features	Multi-semester and multi-course support; letter-grade to grade-point conversion (A–F on a 4.0 scale); quality-points calculation; semester-wise and overall CGPA reporting; full input validation

The program is structured around a `Course` struct (name, grade, credit hours, grade point, quality points) and a set of reusable input-validation functions. For each semester, the program collects the number of courses and then each course's details, converts each letter grade to a numeric grade point, and calculates quality points as $\text{grade point} \times \text{credit hours}$. The semester GPA is the sum of quality points divided by the sum of credit hours for that semester; the overall CGPA applies the same formula across all semesters combined.

```

===== Semester 1 Report =====
Course      Grade  Credit Hrs  Quality Pts
-----
DSA          A      4          16
Maths        B      3           9
-----
Semester GPA: 3.57
Overall CGPA : 3.57 / 4.00

```

Testing: the program was compiled with `g++ -std=c++17 -Wall` with zero warnings, and tested with valid multi-course input, invalid grade letters, non-numeric credit hours, and zero-course edge cases — all handled correctly with re-prompts rather than crashes.

3.7.2 Banking System – Architecture

Key C++ Concepts	Object-oriented programming (classes, encapsulation, composition), const-correctness, vectors of objects, ctime for timestamps, menu-driven program design
Core Features	Auto-generated customer IDs and account numbers; deposit/withdraw with balance protection; fund transfer between accounts; timestamped transaction history; full customer listing

The system is modeled with four classes: Transaction (a single money-movement record with type, amount, and timestamp), Account (holds a balance and a list of Transactions, and exposes deposit/withdraw operations), Customer (wraps a name, ID, and one Account), and Bank (manages the full list of customers and exposes the menu-driven operations: create account, deposit, withdraw, transfer, view info, view history). Withdrawals and transfers are protected against overdrawing an account by checking the balance before any deduction.

```

Account created successfully!
Customer ID: 1 | Account Number: 1001

Transfer successful!
New balance of Account 1001: 3000.00

Transaction History for Account 1001:
Type      Amount    Date/Time
Deposit    5000.00   18-08-2026 20:12:38
Transfer-Out 2500.00   18-08-2026 20:12:38

```

Testing note: during testing, a real bug was found where fund transfers were being logged twice in the transaction history (once as a generic Withdrawal/Deposit, and again as a Transfer-Out/Transfer-In). This was fixed by introducing dedicated `withdrawForTransfer()` and `depositForTransfer()` methods used only in the transfer

path, so each transfer is now logged exactly once per account. This is discussed further in Section 3.8 (Methodology).

3.7.3 Sudoku Solver – Architecture

Key C++ Concepts	2D arrays, recursion, backtracking algorithm design, pass-by-reference, nested-loop constraint checking
Core Features	2D-array grid representation; full rule validation (row/column/3×3 box) before every placement; detection of an already-invalid starting puzzle; detection of unsolvable puzzles; built-in sample puzzle for quick demonstration

The puzzle is stored as a 9×9 2D array, with 0 representing an empty cell. The solver repeatedly finds the next empty cell and tries placing digits 1 through 9 in it. Before each placement, it checks that the digit does not already appear in the same row, column, or 3×3 subgrid. If a digit is valid, it is placed and the algorithm recursively attempts to solve the rest of the grid. If a later cell cannot be filled validly, the algorithm backtracks — it undoes the most recent placement and tries the next candidate digit — until the entire grid is either solved or proven unsolvable.

Original Puzzle:	Solved Puzzle:
5 3 . . 7	5 3 4 6 7 8 9 1 2
6 . . 1 9 5 . . . -->	6 7 2 1 9 5 3 4 8
. 9 8 6 .	1 9 8 3 4 2 5 6 7
(... full 9x9 grid solved correctly ...)	

Testing: the solver was tested on a classic, well-known Sudoku puzzle and produced the correct, verified solution. It was also tested with a deliberately invalid starting grid (a duplicate number in one row), which the program correctly detected and rejected before attempting to solve.

3.8 Methodology

Development Methodology

Each project followed the same disciplined process: design the data structures and functions first, implement with careful input validation throughout, compile with warnings enabled (g++ -Wall) to catch issues early, and then actually run the compiled program with realistic sample input to verify the output — rather than assuming the code was correct without testing.

Testing and Debugging — Challenges Faced and Solutions Implemented

Challenge 1 – Duplicate Transaction Logging in the Banking System: while testing the fund-transfer feature, it was discovered that each transfer was being recorded twice in an account's transaction history — once as a generic Deposit/Withdrawal, and again as a Transfer-In/Transfer-Out — because the transfer function was calling the standard deposit()/withdraw() methods (which log their own entry) and then separately logging a transfer entry. Solution: two dedicated methods, withdrawForTransfer() and depositForTransfer(), were introduced and used only within the transfer operation, so a transfer is now logged exactly once per account. This was verified by re-running the same test scenario and confirming a single, correct log entry.

Challenge 2 – Input Validation Against Invalid or Non-Numeric Input: across all three projects, a recurring challenge was preventing the program from crashing or entering an infinite loop when the user entered unexpected input (letters instead of numbers, out-of-range values, empty strings). Solution: reusable input-validation helper functions were written for each project (e.g. readPositiveInt(), readValidGrade(), readNonNegativeDouble()) that check the input stream's failure state, clear it, and re-prompt the user until valid input is provided.

Challenge 3 – Validating an Already-Invalid Sudoku Puzzle: early testing raised the question of what should happen if a user enters a starting Sudoku grid that already violates the rules (e.g. a repeated number in one row) — the backtracking algorithm alone would simply report “no solution,” which would be misleading. Solution: a separate isInitialGridValid() check was added to run before the solver, so the program can clearly distinguish between “your input is invalid” and “this valid puzzle has no solution.”

3.9 Expected Outcomes

The internship was expected to result in three fully working, independently tested C++ console applications, along with the practical skills and documentation needed to present them as a professional portfolio. The following outcomes were achieved:

Learning Outcomes

- Gained practical experience translating written requirements (the CodeAlpha task brief) into a working technical implementation.
- Strengthened understanding of object-oriented design by modeling a real-world domain (banking) using classes, encapsulation, and composition.
- Learned to implement and explain a recursive backtracking algorithm from scratch.
- Developed the habit of validating all user input rather than assuming well-formed input.
- Learned the importance of actually compiling and running code with real test cases before considering a task complete — a real duplicate-logging bug was found this way in the Banking System and fixed.
- Practiced preparing a project for public submission: structuring a GitHub repository, writing a clear README, and using .gitignore correctly.

Technical Skills Developed

- C++ programming (C++17): structs, classes, functions, vectors, recursion
- Object-oriented programming: encapsulation, composition, const-correctness
- Algorithm design: recursive backtracking
- Input validation and defensive programming
- Debugging and root-cause testing
- Git and GitHub for version control and project submission

Soft Skills Developed

- Requirement analysis and following a specification precisely
- Structured problem-solving and systematic testing
- Clear technical communication through documentation

- Time management across multiple parallel project deliverables

Overall Outcome

This internship provided practical, hands-on experience in C++ programming through three complete projects: a CGPA Calculator demonstrating core procedural programming, a Banking System demonstrating object-oriented design, and a Sudoku Solver demonstrating recursive algorithm design. All three tasks fulfilled CodeAlpha's stated requirements, were compiled without warnings, and were verified through actual test runs with documented sample input and output. Beyond the code itself, the internship reinforced the importance of careful requirement analysis, thorough input validation, genuine testing, and clear technical documentation — skills that extend well beyond this specific internship into future software development work.

Future Scope

- CGPA Calculator: support +/- letter grades and alternative grading scales; add persistent storage of student records across runs.
- Banking System: persist customer and account data to a file or database so it survives program restarts; support multiple accounts per customer; add PIN/password protection.
- Sudoku Solver: add an optional graphical interface for visual puzzle entry and display, as suggested (optionally) in the original task description.
- General: extend the portfolio by completing the remaining task (Login and Registration System) to cover all four CodeAlpha C++ tasks.

3.10 Offer Letter and Other Communication Proof

The official CodeAlpha Internship Offer Letter for Aditya Agrawal is reproduced in full immediately after the Table of Contents of this report. A summary of the verifiable offer letter details is provided below for quick reference. No additional communication/email proof beyond the offer letter and the official internship task document was issued by the organization.

Table 3.3: Offer Letter Verification Summary

Certified Name	Aditya Agrawal
Program	C++ Programming Internship
Internship Duration	20 August 2026 to 20 September 2026
Offer Letter Date	12 August 2026
Student ID	CA/DF1/245935
Signing Authority	Swati Srivastava, Founder & CEO, CodeAlpha

4. Bibliography / References

- [1] CodeAlpha – C++ Programming Tasks & Instructions (official internship task document), www.codealpha.tech
- [2] C++ Standard Library Documentation, <https://cppreference.com>
- [3] CodeAlpha Internship Offer Letter, Student ID: CA/DF1/245935, issued 12 August 2026.
- [4] Project source code and documentation repositories on GitHub — [TO BE FILLED: insert GitHub repository links].